

openagent-infra

OpenAgent model inference infrastructure — the model serving proxy layer of the OpenAgent system.

Overview

`openagent-infra` is the inference proxy for OpenAgent. This repo is solely responsible for proxying requests to external compute providers, authenticating callers, and streaming responses via a small, production-shaped REST API.

It is intentionally scoped to the model layer only. It has no knowledge of the OpenAgent persona, the frontend, or the conversation state — those live in separate repos. The boundary is clean by design: **openagent-infra proxies the models, everything else builds on top of it.**

The BYOC strategy

`openagent-infra` is deliberately model- and provider-agnostic, built around a **Bring Your Own Compute (BYOC)** approach. It proxies requests to any OpenAI-compatible API endpoint — vLLM workers on RunPod, a local Ollama instance, a standard commercial API, whatever you point it at.

It routes between two logical models:

- **base_model** — the primary reasoning model handling everyday conversations.
- **nervous_system** — the fast, lightweight control layer handling routing, history filtering, and agent decisions.

Keeping the proxy provider-agnostic means the rest of the stack never has to care where inference actually happens; you can swap providers or model sizes by changing two URLs and a key.

Where This Fits

```
openagent-os
├── openagent-infra    ← YOU ARE HERE
│   ├── Model proxy API (port 8002)
│   └── Model proxy layer
├── openagent-frontend ← separate repo
│   ├── The product experience (port 8000)
│   └── Talks to openagent-api
├── openagent-api      ← separate repo
│   ├── The Identity Gateway (port 8001)
│   └── Talks to openagent-infra
```

└─ openagent-logger ← separate repo
└─ The capture layer (port 8003)

The naming convention is intentional:

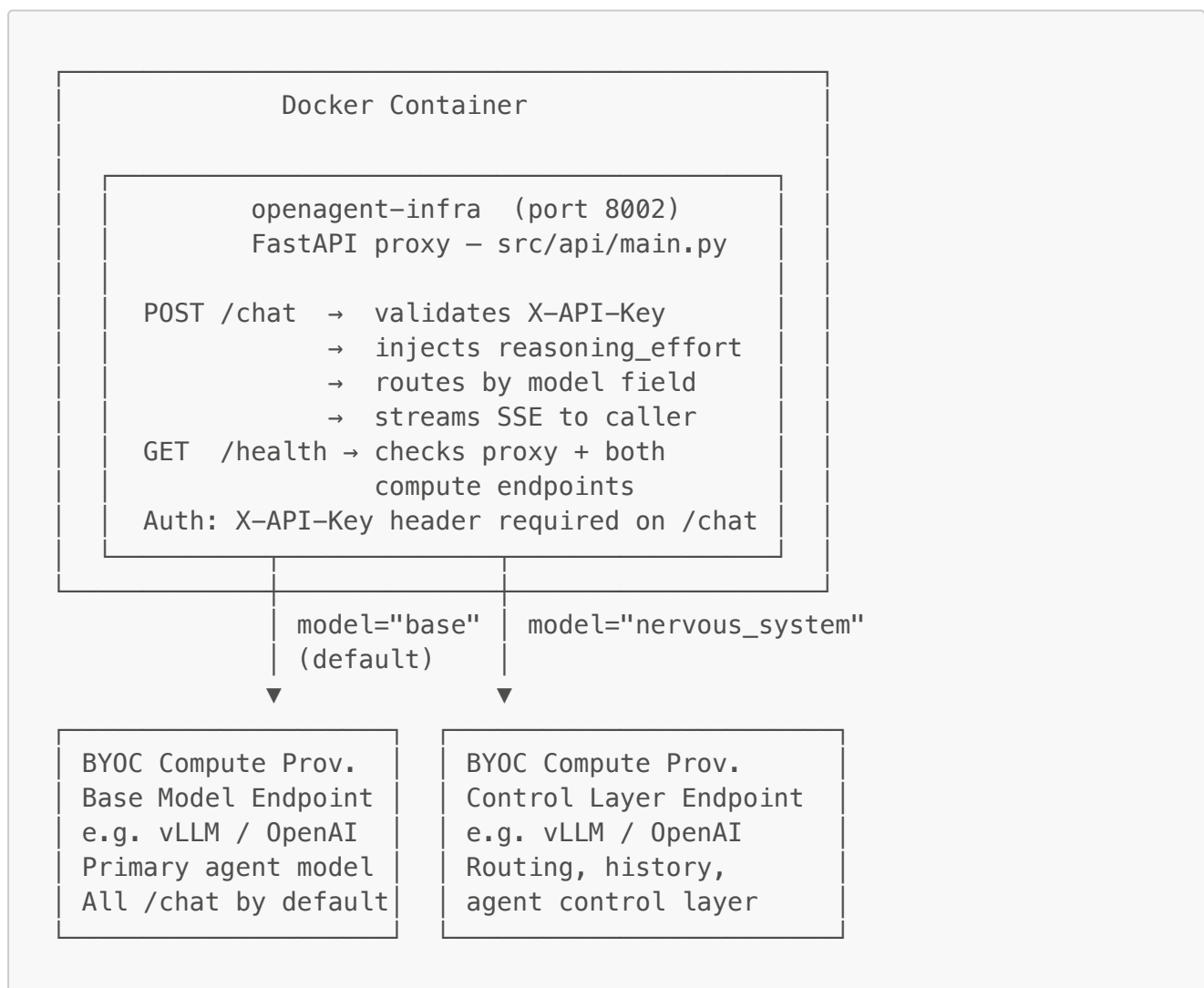
- **openagent-infra** handles the **model** connectivity and compute provision.
- **openagent-*** (api, frontend, logger) handle the **product** — gateway, UI, identity, and state.

Port topology:

```
openagent-api (:8001) → openagent-infra (:8002) → BYOC Provider Base Model [default]
                                                                → BYOC Provider Control Layer Model [model="nervous_system"]
```

openagent-infra is accessed exclusively by openagent-api.

Architecture



Request flow

1. `openagent-api` sends `POST /chat` with `X-API-Key`, messages list, optional `reasoning_effort`, and optional `model`.
2. `openagent-infra` validates the API key — returns `401` if missing or invalid.
3. `openagent-infra` injects `Reasoning: <level>` into the system message automatically.
4. `openagent-infra` routes to the correct external endpoint — base model by default, control layer when `model="nervous_system"`.
5. `openagent-infra` forwards via `htpx` with `Authorization: Bearer PROVIDER_API_KEY`.
6. The BYOC provider generates tokens.
7. Tokens stream back through `openagent-infra` to the caller as SSE events.
8. A final `data: [DONE]` event signals end of stream.

System prompt ownership

The system prompt — the persona — is owned upstream by `openagent-api`. `openagent-api` sends it as the first message in the OpenAI messages list on every request. `openagent-infra` injects the reasoning effort level into it automatically before forwarding to the compute provider. `openagent-infra` never stores or inspects the system prompt content.

Tech Stack

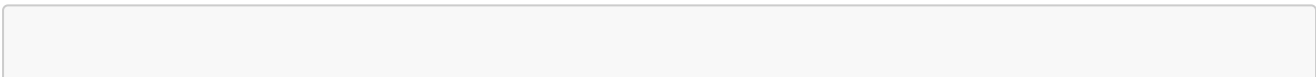
Layer	Technology
Base image	<code>python:3.12-slim</code>
Model serving	BYOC — any OpenAI-compatible API endpoint
API proxy	FastAPI + uvicorn
Streaming	SSE via <code>htpx</code> async proxy
Auth	<code>X-API-Key</code> header (caller) + <code>PROVIDER_API_KEY</code> Bearer (Compute Provider)
Containerization	Docker + Docker Compose

Prerequisites

- **Docker Desktop** installed
- **A compute provider** (e.g., RunPod, OpenAI, local Ollama) serving two endpoints.
- **PROVIDER_API_KEY** — an API key with access to both compute endpoints.
- **API_KEY** — a secret key shared with `openagent-api` for request authentication.

No local GPU required unless you are self-hosting your BYOC endpoints locally.

Project Structure



```
openagent-infra/
├── docker/
│   └── model/
│       └── Dockerfile                # python:3.12-slim – proxy only, no
CUDA
├── src/
│   └── api/
│       └── main.py                  # FastAPI proxy – auth, routing,
reasoning injection
├── docker-compose.yml
├── requirements.txt
├── .env                            # secrets – never commit this
├── .env.example                    # template for .env
├── .dockerignore
├── .gitignore
└── README.md
```

Setup

1. Clone the repo

```
git clone https://github.com/william-mckeon/openagent-infra.git
cd openagent-infra
```

2. Create your `.env` file

```
cp .env.example .env
```

Edit `.env` and fill in your values:

```
API_KEY=your_long_random_secret_key_here
BASE_MODEL_URL=https://your-provider.com/v1/chat/completions
NERVOUS_SYSTEM_URL=https://your-provider.com/v1/chat/completions
PROVIDER_API_KEY=your_provider_api_key_here
REASONING_EFFORT=medium
```

Generate a secure `API_KEY` with:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

3. Build the image

```
docker-compose build --no-cache
```

4. Start the API proxy

```
docker-compose up -d
```

The API is ready when you see:

```
=== OpenAgent Inference API Ready – listening on :8002 ===  
INFO:      Application startup complete.  
INFO:      Uvicorn running on http://0.0.0.0:8002
```

Startup takes under 10 seconds — the proxy has no model to load.

API Reference

POST /chat

Send a full OpenAI messages list and receive a streamed response via Server-Sent Events. Optionally set the reasoning effort level and model per request.

Requires a valid **X-API-Key** header on every request.

Request headers:

```
Content-Type: application/json  
X-API-Key: your_api_key_here
```

Request body:

```
{  
  "messages": [  
    {"role": "system",    "content": "You are OpenAgent..."},  
    {"role": "user",     "content": "hello"}  
  ],  
  "reasoning_effort": "medium",  
  "model": "base"  
}
```

Field	Type	Required	Description
<code>messages</code>	array	Yes	Full OpenAI messages list including system prompt
<code>reasoning_effort</code>	string	No	<code>low</code> , <code>medium</code> , or <code>high</code> . Defaults to <code>medium</code> .
<code>model</code>	string	No	<code>base</code> (default) or <code>nervous_system</code> . Routes to the base endpoint or the control layer endpoint.

Error responses:

- `400` — messages list is empty or contains no user message
- `401` — API key missing or invalid
- `422` — request body malformed
- `503` — compute endpoint not reachable

curl:

```
curl -X POST http://localhost:8002/chat \
  -H "Content-Type: application/json" \
  -H "X-API-Key: your_api_key_here" \
  -d '{"messages": [{"role": "system", "content": "You are OpenAgent..."}, {"role": "user", "content": "hello"}], "reasoning_effort": "medium"}' \
  --no-buffer
```

GET /health

Health check. No authentication required. Checks both the proxy and both compute endpoints independently.

Fully ready:

```
{"status": "ok", "proxy": "ok", "base_model": "ok", "nervous_system": "ok"}
```

base_model unreachable / cold-starting:

```
{"status": "degraded", "proxy": "ok", "base_model": "unreachable", "nervous_system": "ok"}
```

Nervous system not yet configured:

```
{"status": "ok", "proxy": "ok", "base_model": "ok", "nervous_system": "not configured"}
```

Status is **ok** when the base model is reachable — nervous-system is checked independently. **not configured** means **NERVOUS_SYSTEM_URL** is not set in **.env**.

```
curl http://localhost:8002/health
```

GET /docs

Auto-generated Swagger UI:

```
http://localhost:8002/docs
```

Configuration

Variable	Default	Description
API_KEY	—	Secret key for X-API-Key auth (required)
BASE_MODEL_URL	—	API endpoint for the primary model. Default for all requests (required)
NERVOUS_SYSTEM_URL	—	API endpoint for the fast control model. Used when model="nervous_system"
PROVIDER_API_KEY	—	API key for Bearer auth on both compute endpoints (required)
REASONING_EFFORT	medium	Default reasoning level — low , medium , or high

Design Decisions

Why keep the FastAPI proxy layer instead of calling providers directly?

Compute providers usually support only a single shared API key. By keeping FastAPI as a proxy, auth validation lives in one place and can be swapped or managed without exposing the provider's billing API key to the gateway or frontend layers.

Why two separate API keys?

API_KEY is the key **openagent-api** sends to **openagent-infra** — it identifies and authenticates the caller. **PROVIDER_API_KEY** is the key **openagent-infra** sends to the compute provider — it

authenticates `openagent-infra` to the inference backend. These concerns are deliberately separated so the caller key can be rotated without affecting the provider configuration, and vice versa.

Why reasoning effort as an API parameter?

Both models support configurable reasoning effort — low, medium, high. It's a genuine feature: a frontend can expose it as Quick / Standard / Deep mode, and tooling can set it per use case programmatically. The proxy injects it into the system message automatically.

Why OpenAI messages format?

The OpenAI messages format makes `openagent-infra` compatible with almost any frontend and model backend on the market. It keeps the serving layer stateless and the protocol standard.

Why port 8002?

Port 8000 is reserved for openagent-frontend. Port 8001 is reserved for openagent-api. Port 8002 is the exposed port for openagent-infra.

License

Copyright © 2026 William McKeon.

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

Maintainer

William McKeon (github.com/william-mckeon)